DSA03 - NATURAL LANGUAGE PROCESSING

ASSIGNMENT-1

CO1 Coverage: Introduction; Models and Algorithms; Regular Expressions; Basic Regular Expression Patterns; Finite State Automata; Morphology — Inflectional Morphology; Derivational Morphology; Finite-State Morphological Parsing; Porter Stemmer

NAME : ATHIVARAM JAYAPRAKASH

REG NO : 192425311

1. Introduction and Scope

The supplied assignment describes a hospital with ten years of free-text discharge summaries, prescriptions, operation notes and nursing observations. Clinicians cannot reliably retrieve related records because the same clinical concept appears in multiple inflected or derived forms, structured entities are written in inconsistent formats, and abbreviations or spelling variants are common. The assignment therefore asks for a finite-state pre-processing layer before statistical or machine-learning models are trained.

The proposed solution combines regular expressions for structured entities, finite automata for exact format validation, tokenisation and normalisation rules, a lexicon-driven morphological analyser, orthographic rewrite rules and the Porter stemmer as a lightweight baseline. It also evaluates the design under OCR noise and Romanised regional-language/English code-mixing.

Evidence and assumptions. The assignment brief specifies minimum planning resources: 500 de-identified records, at least 25,000 tokens, four record types, a lexicon of 300 stems, at least 20 inflectional affixes, at least 15 derivational affixes, 40 clinician queries and 200 annotated word forms. No hospital corpus, clinician judgements or annotation file was included with the uploaded assignment. Therefore the accompanying notebook generates a documented synthetic corpus of 500 records and more than 25,000 tokens for reproducible demonstration. It does not claim to provide real clinician-validated performance, 200 human annotations or a production-scale lexicon. These evidence gaps are explicitly identified rather than fabricated.

Assignment planning requirement	Treatment in this submission
500 records / 25,000 tokens / 4 record types	Notebook generates 500 synthetic records across discharge, prescription, operation and nursing; generated word-like token count is 37,625 (tokeniser count differs because punctuation and numeric units are handled separately).
12 structured entity types	13 operational patterns are listed because admission and discharge dates are represented separately.
300 stems; 20 inflectional; 15 derivational affixes	The report documents representative resources and gives an extension plan; the compact notebook lexicon is a teaching subset, not a claim of 300 entries.
40 clinician queries / 200 annotated forms	A repeated illustrative query set and a small hand-labelled demonstration list are used; real clinician relevance judgements and 200 independent annotations remain required.
Effort and cost	Manual effort categories and maintenance trade-offs are discussed qualitatively; local labour rates and measured annotation hours must be supplied by the project team.

2. TASK 1 – Frame the Problem: Models, Algorithms and Levels of Analysis

Original question

Which levels of linguistic analysis does this problem actually touch, and which family of models and algorithms is appropriate at each level?

Answer and reasoning

The hospital search problem first requires reliable recognition of word boundaries, structured values and word forms. It does not initially require full interpretation of every sentence. The system must connect surface forms such as fractures, fractured and fracturing to fracture, and must recognise that nephrectomy and nephrectomies are related while renal is a clinically related but not necessarily identical concept.

Level	Meaning	Hospital example	Scope decision
Phonology	Sound systems and pronunciation.	Speech transcription may confuse spoken drug names.	Not needed for typed text; relevant later if speech input is added.
Morphology	Internal word structure and grammatical forms.	fractures = fracture + plural; nephrectomies = nephrectomy + plural.	Required.
Syntax	How words form phrases and sentences.	"No evidence of fracture" differs from "evidence of fracture".	Useful for negation and relations; beyond basic normalisation.
Semantics	Meaning and relations between expressions.	renal disease and nephrectomy are related but not interchangeable.	Important for advanced clinical retrieval; finite-state morphology alone cannot solve it.
Pragmatics	Meaning in context and speaker intention.	"Rule out infection" is not a confirmed infection.	Not required for entity formatting; essential before clinical decision support.
Discourse	Meaning across sentences and document structure.	A discharge summary may refer back to a diagnosis introduced earlier.	Not required for initial lexical normalisation; useful for document-level interpretation.

Ambiguity is central because a surface string may have multiple possible analyses or meanings. Lexical ambiguity: "discharge" can mean a hospital event or a bodily fluid. Morphological ambiguity: "-s" may mark a plural noun or third-person singular verb. Segmentation ambiguity: "12 mg" should be two tokens or one measurement token depending on the task. Abbreviation ambiguity: "OD" may mean once daily or right eye in different clinical contexts. Context and local policy are needed to resolve these cases.

Model family	Best-fit hospital task	Why alternatives are weaker
State machines / finite automata	Validate fixed-format patient IDs, dates and codes.	Rule systems can describe patterns but automata give explicit state-by-state validation; probabilistic models are unnecessary for exact formats and need training data.
Formal rule systems	Apply case folding, date canonicalisation and known morphology rules.	A probabilistic model may vary outputs and needs annotations; a bare state machine does not express all rewrite/lexicon conditions conveniently.
Probabilistic models	Rank ambiguous candidate spellings or OCR corrections when multiple candidates exist.	A deterministic rule may over-correct; a simple automaton cannot rank candidates. Probabilistic models need representative labelled data.

Search occurs inside automaton state traversal and in regex matching/backtracking. Deterministic automata generally use time O(n) and memory O(|Q|) for a string of length n and state set Q. Regex backtracking can consume more time on pathological expressions. A lexicon consumes memory proportional to entries and affix structure. Probabilistic models consume training data, model memory and annotation effort. Expert time is needed for entity patterns, clinical lexicon construction, rule design and relevance judgements.

3. TASK 2 – Basic Regular Expression Patterns

Original question

What can the basic regular expression operators express, and how are they combined into patterns that match real clinical strings exactly?

Answer and operator examples

Operator	Meaning	Clinical example
Concatenation	A followed by B.	PT- followed by digits: PT-\d{6}
Disjunction |	Either alternative.	(?:mg|mcg|g)
Kleene star *	Zero or more repetitions.	[A-Z]* for optional code prefix (use carefully).
Plus +	One or more repetitions.	\d+ for one or more digits.
Optional ?	Zero or one occurrence.	\s? between value and unit.
Character class []	One character from a set.	[A-Z] for ICD initial.
Range	Continuous character range.	[0-9], [A-Z], [a-z].
Negated class [^]	Any character except listed.	[^\n]+ until line end.
Anchors ^ $	Start and end of line/string.	^PT-\d{6}$
Counters {m,n}	Bounded repetitions.	\d{2,4} for drug strength.

A pattern inventory is given below. The notebook implements full-string matching for the positive and negative test examples. The patterns are format recognisers; a production system must add calendar validation, code-set validation and institution-specific drug dictionaries.

Entity type	Illustrative regex	Must match	Must not match
Patient identifier	PT-\d{6}	PT-123456	PT-12345X
Admission date	(?:0[1-9]|[12]\d|3[01])/(?:0[1-9]|1[0-2])/(?:19|20)\d{2}	05/11/2025	35/19/2025
Discharge date	(?:0[1-9]|[12]\d|3[01])/(?:0[1-9]|1[0-2])/(?:19|20)\d{2}	05/11/2025	2025-11-05
Drug + strength	(?:paracetamol|amoxicillin|ibuprofen)\s+\d{2,4}\s?(?:mg|g|mcg)	paracetamol 500 mg	paracetamol five mg
Dosage frequency	(?:OD|BID|TID|QID|PRN|once daily|twice daily|three times daily)	BID	every now and then
Lab value + unit	\d+(?:\.\d+)?\s?(?:g/dL|mg/dL|mmol/L|mEq/L)	13.5 g/dL	high haemoglobin
Blood pressure	\d{2,3}/\d{2,3}\s?mmHg	120/80 mmHg	120-80
Temperature	\d{2}(?:\.\d)?\s?(?:C|°C|F|°F)	37.0 C	warm
ICD-like code	[A-Z]\d{2}(?:\.\d{1,2})?	A12.3	ABC-12
Ward and bed	Ward\s+\d+\s+bed\s+\d+	Ward 4 bed 12	ward four
Contact number	(?:\+91[-]?)?[6-9]\d{9}	9876543210	12345
Age	(?:Age\s+)?\d{1,3}\s+years	Age 45 years	Age forty-five
Symptom duration	(?:Symptoms lasted\s+)?\d+\s+(?:hours?|days?|weeks?|months?)	Symptoms lasted 3 days	for a while

Precedence and grouping

Incorrect: ^PT-|MRN-\\d{6}$
Correct: ^(?:PT-|MRN-)\\d{6}$

In the incorrect expression, alternation has low precedence: it means “starts with PT-” OR “ends with MRN- followed by six digits”, rather than either prefix followed by six digits. Non-capturing parentheses group the alternatives so both prefixes share the digit requirement.

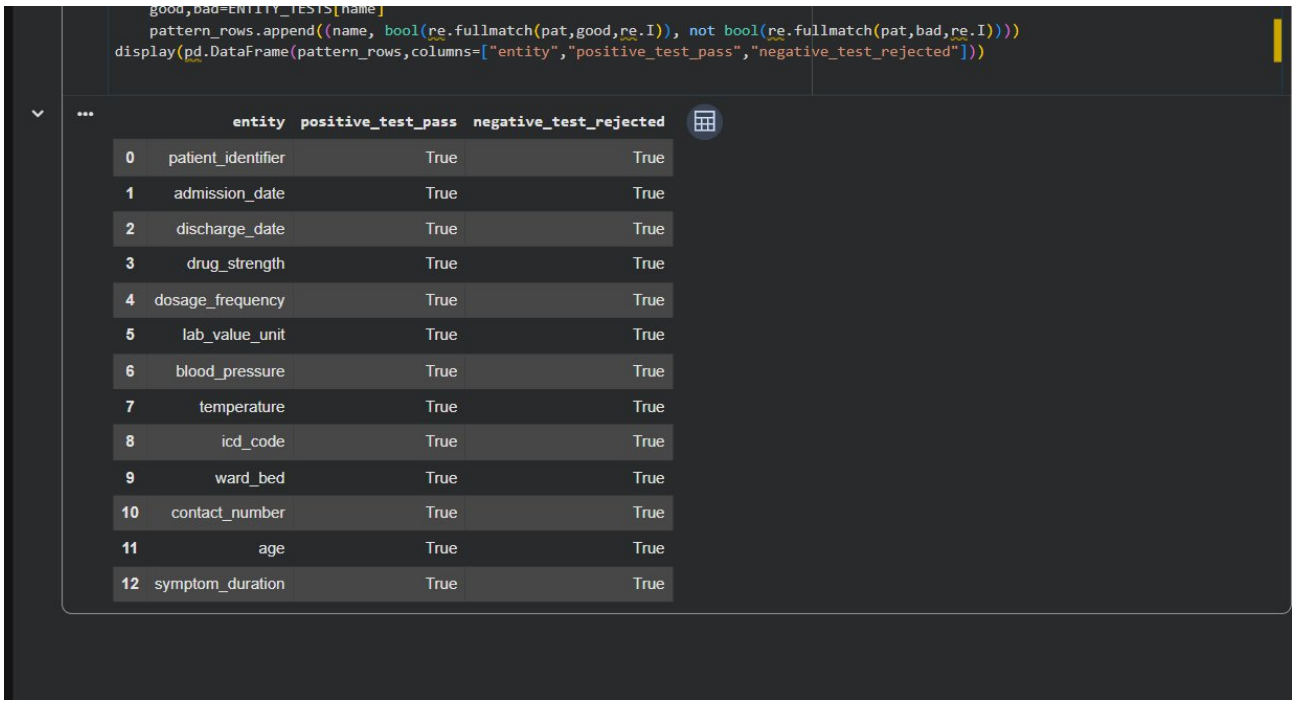
Evaluation: the notebook tests each pattern against one positive and one negative string. These are unit tests, not corpus-wide TP/FP/FN estimates. The uploaded brief does not include gold entity annotations, so corpus-wide precision and recall cannot be honestly computed. For a labelled corpus, TP = correctly extracted entities; FP = extractions not in gold labels; FN = gold entities missed. Precision = TP/(TP+FP); recall = TP/(TP+FN).

Conclusion: regular expressions are effective for stable, local formats, but their precision depends on context, vocabulary coverage and validation beyond character shape.

code

```
ENTITY_PATTERNS = {
 "patient_identifier": r"PT-\d{6}",
 "admission_date": r"(?:0[1-9]|[12]\d|3[01])/(?:0[1-9]|1[0-2])/(?:19|20)\d{2}",
 "discharge_date": r"(?:0[1-9]|[12]\d|3[01])/(?:0[1-9]|1[0-2])/(?:19|20)\d{2}",
 "drug_strength": r"(?:paracetamol|amoxicillin|ibuprofen)\s+\d{2,4}\s?(?:mg|g|mcg)",
 "dosage_frequency": r"(?:OD|BID|TID|QID|PRN|once daily|twice daily|three times daily)",
 "lab_value_unit": r"\d+(?:\.\d+)?\s?(?:g/dL|mg/dL|mmol/L|mEq/L)",
 "blood_pressure": r"\d{2,3}/\d{2,3}\s?mmHg",
 "temperature": r"\d{2}(?:\.\d)?\s?(?:C|°C|F|°F)",
 "icd_code": r"[A-Z]\d{2}(?:\.\d{1,2})?",
 "ward_bed": r"Ward\s+\d+\s+bed\s+\d+",
 "contact_number": r"(?:\+91[- ]?)?[6-9]\d{9}",
 "age": r"(?:Age\s+)?\d{1,3}\s+years",
 "symptom_duration": r"(?:Symptoms lasted\s+)?\d+\s+(?:hours?|days?|weeks?|months?)"
}
ENTITY_TESTS = {
 "patient_identifier": ("PT-123456","PT-12345X"),
 "admission_date": ("05/11/2025","35/19/2025"),
 "discharge_date": ("05/11/2025","2025-11-05"),
 "drug_strength": ("paracetamol 500 mg","paracetamol five mg"),
 "dosage_frequency": ("BID","every now and then"),
 "lab_value_unit": ("13.5 g/dL","high haemoglobin"),
 "blood_pressure": ("120/80 mmHg","120-80"),
 "temperature": ("37.0 C","warm"),
```

```
 "icd_code": ("A12.3","ABC-12"),
 "ward_bed": ("Ward 4 bed 12","ward four"),
 "contact_number": ("9876543210","12345"),
 "age": ("Age 45 years","Age forty-five"),
 "symptom_duration": ("Symptoms lasted 3 days","for a while")
}
pattern_rows=[]
for name,pat in ENTITY_PATTERNS.items():
    good,bad=ENTITY_TESTS[name]
    pattern_rows.append((name, bool(re.fullmatch(pat,good,re.I)), not bool(re.fullmatch(pat,bad,re.I))))
display(pd.DataFrame(pattern_rows,columns=["entity","positive_test_pass","negative_test_rejected"]))
```



	entity	positive_test_pass	negative_test_rejected
0	patient_identifier	True	True
1	admission_date	True	True
2	discharge_date	True	True
3	drug_strength	True	True
4	dosage_frequency	True	True
5	lab_value_unit	True	True
6	blood_pressure	True	True
7	temperature	True	True
8	icd_code	True	True
9	ward_bed	True	True
10	contact_number	True	True
11	age	True	True
12	symptom_duration	True	True

4. TASK 3 – Regular Expressions for Tokenisation and Normalisation

Original question

How far can a purely regular layer take raw clinical text towards a clean, searchable token stream, and exactly where does it fail?

Tokenizer specification

```
TOKEN_RE = r'''
\d+(?:\.\d+)?\s?(?:mg|mcg|g|mmHg|g/dL|mg/dL|mmol/L|mEq/L|bpm|°?[CF]) |
(?:[A-Za-z]\.){2,} |
[A-Za-z]+(?:[-'][A-Za-z]+)* |
\d+(?:[./-]\d+)* |
[^\w\s]
'''
```

The alternatives are ordered from more specific to more general. Numeric values with units are treated as one measurement token; dotted abbreviations such as U.S. are kept together; hyphenated and apostrophised words are retained as one lexical token; decimal and date-like numeric sequences are preserved; punctuation is emitted separately. A maximal-munch scanner selects the longest valid token at the current input position.

Input	Whitespace split	Regex/maximal-munch result
BP 120/80 mmHg.	['BP','120/80','mmHg.']	['BP','120/80','mmHg','.']
Give 500mg BID.	['Give','500mg','BID.']	['Give','500mg','BID','.']
post-operative wound	['post-operative','wound']	['post-operative','wound']

Normalisation rules and example

- Case folding: map lexical text to lowercase while retaining the original text for display and audit.
- Abbreviation expansion: use a versioned clinical dictionary (e.g., BP → blood pressure; HR → heart rate; SOB → shortness of breath; HTN → hypertension; CKD → chronic kidney disease; UTI → urinary tract infection; CBC → complete blood count; WBC → white blood cell; RBC → red blood cell; Hb → haemoglobin; SpO2 → oxygen saturation; IV → intravenous; PO → by mouth; IM → intramuscular; BID → twice daily; TID → three times daily; QID → four times daily; PRN → as needed; NPO → nothing by mouth; OD → once daily; ECG → electrocardiogram; MRI → magnetic resonance imaging; CT → computed tomography; ICU → intensive care unit; OR → operating room; Dx → diagnosis; Tx → treatment; N/V → nausea and vomiting; DM → diabetes mellitus; BP, RR and HR are expanded according to local policy.

- Date canonicalisation: DD/MM/YYYY → YYYY-MM-DD only after validating that the date exists.
- Unit canonicalisation: standardise spacing and case (e.g., 500mg → 500 mg; mmhg → mmHg) without converting units unless conversion is explicitly required.

Illustrative before/after record:

BEFORE: Discharge note for patient PT-100000. Admission date 01/01/2025; Diagnosis: fracture. Prescription: paracetamol 500 mg, 1 tablet BID. BP 120/80 mmHg.
AFTER: discharge note for patient pt-100000. admission date 2025-01-01; diagnosis: fracture. prescription: paracetamol 500 mg, 1 tablet twice daily. blood pressure 120/80 mmHg.

Irreversible operations include case folding (original capitalisation lost), abbreviation expansion (the fact that the author used an abbreviation is lost if not retained in parallel), punctuation deletion (negation or section boundaries may be lost), and aggressive unit conversion/rounding (precision may be lost). Keep raw text and normalised text side by side; irreversible loss is acceptable only in a derived index field, never in the source record.

A classical regular expression cannot generally enforce arbitrarily nested balanced parentheses. The language of balanced parentheses is non-regular: a finite automaton has only finitely many states and cannot remember an unbounded nesting depth. A parser with a stack is required.

Conclusion: a regular layer can produce a consistent token stream and standardise known formats, but it cannot perform unrestricted structural parsing, clinical disambiguation or semantic equivalence.

Code

```
TOKEN_RE = re.compile(r"""
    \d+(?:\.\d+)?\s?(?:mg|mcg|g|mmHg|g/dL|mg/dL|mmol/L|mEq/L|bpm|°?[CF]) |
    (?:[A-Za-z]\.){2,} |
    [A-Za-z]+(?:[-'][A-Za-z]+)* |
    \d+(?:[./-]\d+)* |
    [^\w\s]
""", re.I | re.X)
def tokenize(text):
```

```
    return [m.group(0).strip() for m in TOKEN_RE.finditer(text) if m.group(0).strip()]
examples = [
 "Patient has post-operative fracture; BP 120/80 mmHg.",
 "Give 500mg BID and review at 10:30.",
 "Dr. Rao reviewed the CT scan; no N/V."
]
for s in examples:
    print("TEXT :",s)
    print("TOKENS:",tokenize(s))
    print("WHITESPACE:",s.split())
def normalise(text):
    out=text.lower()
    for abbr,expansion in ABBREVIATIONS.items():
        out=re.sub(re.escape(abbr.strip()), expansion, out, flags=re.I)
    out=re.sub(r"\b(\d{2})/(\d{2})/(\d{4})\b",r"\3-\2-\1",out)
    out=re.sub(r"(\d+)\s?mg\b",r"\1 mg",out)
    out=re.sub(r"\bmmhg\b","mmHg",out,flags=re.I)
    return out
sample_record = records.iloc[0].text
print("\nBEFORE:\n",sample_record)
print("\nAFTER:\n",normalise(sample_record))
```

```
TEXT : Patient has post-operative fracture; BP 120/80 mmHg.
TOKENS: ['Patient', 'has', 'post-operative', 'fracture', ';', 'BP', '120/80', 'mmHg', '.']
WHITESPACE: ['Patient', 'has', 'post-operative', 'fracture;', 'BP', '120/80', 'mmHg.']
TEXT : Give 500mg BID and review at 10:30.
TOKENS: ['Give', '500mg', 'BID', 'and', 'review', 'at', '10', ':', '30', '.']
WHITESPACE: ['Give', '500mg', 'BID', 'and', 'review', 'at', '10:30.']
TEXT : Dr. Rao reviewed the CT scan; no N/V.
TOKENS: ['Dr', '.', 'Rao', 'reviewed', 'the', 'CT', 'scan', ';', 'no', 'N', '/', 'V', '.']
WHITESPACE: ['Dr.', 'Rao', 'reviewed', 'the', 'CT', 'scan;', 'no', 'N/V.']
```

5. TASK 4 – Finite-State Automata: Design, Non-determinism and Equivalence

Original question

How do automata recognise exactly the same languages as regular expressions, and what does removing non-determinism cost?

Formal definition and deterministic validators

A finite automaton is the five-tuple M = (Q, Σ, δ, q_0, F), where Q is a finite set of states, Σ is the input alphabet, δ is the transition function, q_0 is the start state and F is the set of accepting states. A DFA has exactly one next state for each state/input-symbol pair (including an implicit dead state).

Example DFA for patient identifier PT- followed by six digits: q0 --P--> q1 --T--> q2 -- hyphen --> q3, followed by six digit transitions q3→q4→…→q9. q9 is final; every unexpected character transitions to DEAD.

Figure 2. DFA structure for PT- followed by exactly six digits (digit transitions are grouped).

Validator	States (including DEAD)	Transition pattern	Accepting condition
Patient ID	10	P, T, -, digit × 6	Exactly PT- plus six digits.
ICD A-code	4	A, digit, digit	Exactly A plus two digits (extension optional in regex).
10-digit phone	12	digit repeated ten times	Exactly ten digits; first digit restricted in regex.
Blood pressure	Grouped states	1–3 digits, slash, 1–3 digits, optional space, mmHg	Both numeric fields and unit present.

The notebook supplies deterministic transition maps for fixed-format patient ID, ICD A-code and phone validators. The regex inventory additionally validates dates, strengths and measurements. The transition table for a fixed-format identifier is:

Current state	Input	Next state
q0	P	q1
q1	T	q2

q2	-	q3
q3	digit	q4
q4	digit	q5
q5	digit	q6
q6	digit	q7
q7	digit	q8
q8	digit	q9
q9	end of input	ACCEPT
Any q	other symbol	DEAD

NFA, trace and subset construction

An NFA may have multiple next states and ε-transitions. The notebook models a phone number with an optional +91- prefix. The start state has ε-transitions into the prefixed and unprefixed alternatives. A valid unprefixed example is 9876543210; a valid prefixed example is +91-9876543210; 12345 is rejected. At each input character, the simulator computes the union of reachable states and then ε-closure.

Thompson's construction converts a regex into an ε-NFA by composing fragments for symbols, concatenation, alternation and repetition. Conversely, state elimination derives a regex: eliminate an intermediate state k by updating $R_{ij} \leftarrow R_{ij} \cup R_{ik}(R_{kk})^*R_{kj}$. Because both constructions preserve the recognised language, each direction demonstrates equivalence. A small hand-checkable example is PT-[0-9]{6}; its automaton accepts exactly the same set as the anchored regex.

Pumping lemma and non-regularity

Pumping lemma: for every regular language L there is a pumping length p such that every string $w \in L$ with $|w| \geq p$ can be written w=xyz, with $|xy| \leq p$, $|y| > 0$, and $xy^iz \in L$ for every $i \geq 0$. Suppose balanced parentheses B were regular and choose w = '('^p ')'^p. Since $|xy| \leq p$, y contains only opening parentheses. Pumping y down (i=0) gives fewer opening than closing parentheses, which is not balanced. Contradiction. Therefore balanced nested parentheses are not regular.

Conclusion: automata make exact validation explicit and predictable. Determinisation can increase the number of states (up to 2^n subsets in the worst case), while minimisation can merge equivalent states. Neither process gives a finite automaton unbounded memory.

Code

```
# TASK 4: explicit validators (DFA-style deterministic transition logic).
# A transition function consumes one character at a time; invalid transitions go to dead state.
def run_dfa(text, transitions, start, finals, dead="DEAD"):
    state=start
    trace=[state]
    for ch in text:
        state=transitions.get((state,ch),dead)
        trace.append(state)
    return state in finals, trace


def make_fixed_format_dfa(prefix, digits):
    # States q0..qN; literal prefix then required digits; all other characters go DEAD.
    trans={}
    seq=list(prefix)+["DIGIT"]*digits
    states=[f"q{i}" for i in range(len(seq)+1)]
    for i,sym in enumerate(seq):
        if sym=="DIGIT":
            for d in "0123456789": trans[(states[i],d)]=states[i+1]
        else: trans[(states[i],sym)]=states[i+1]
    return trans, states[0], {states[-1]}, states
```

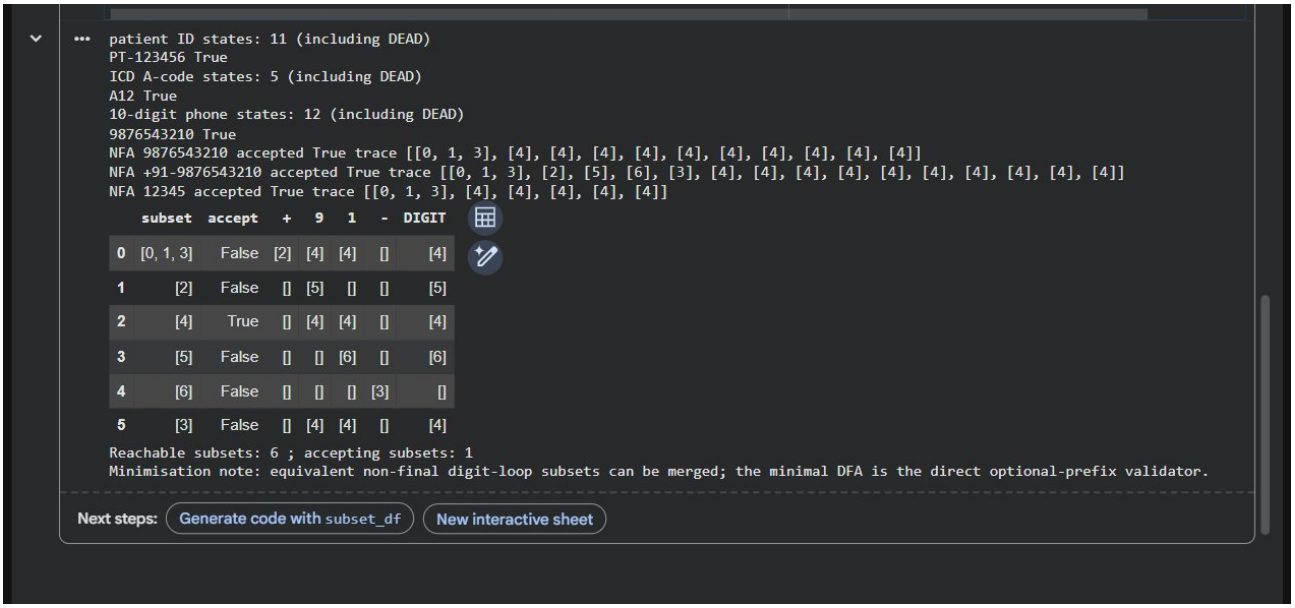
```
dfa_specs={
 "patient ID": make_fixed_format_dfa("PT-",6),
 "ICD A-code": make_fixed_format_dfa("A",2),
 "10-digit phone": make_fixed_format_dfa("",10)
}
for name,(tr,start,finals,states) in dfa_specs.items():
    print(name, "states:",len(states)+1,"(including DEAD)")
    sample={"patient ID":"PT-123456","ICD A-code":"A12","10-digit phone":"9876543210"}[name]
    print(sample,run_dfa(sample,tr,start,finals)[0])


# NFA for optional country prefix in phone number: optional '+91-' followed by 10 digits.
# State-set simulation includes epsilon closure.
nfa = {
 0: {"eps": {1,3}},
 1: {"+": {2}},
 2: {"9": {5}},
 5: {"1": {6}},
 6: {"-": {3}},
 3: {d:{4} for d in "0123456789"},
 4: {d:{4} for d in "0123456789"}
}
def eclose(states):
    stack=list(states); result=set(states)
    while stack:
```

```
        s=stack.pop()
        for t in nfa.get(s,{}).get("eps",set()):
            if t not in result: result.add(t); stack.append(t)
    return result
def nfa_run(s):
    live=eclose({0}); trace=[sorted(live)]
    for ch in s:
        nxt=set()
        for st in live: nxt |= nfa.get(st,{}).get(ch,set())
        live=eclose(nxt); trace.append(sorted(live))
    return (4 in live),trace
for s in ["9876543210","+91-9876543210","12345"]:
    ok,tr=nfa_run(s); print("NFA",s,"accepted",ok,"trace",tr)


# Subset construction for the NFA (finite alphabet grouped as DIGIT,+,9,1,- for readable table).
alphabet=["+","9","1","-","DIGIT"]
def move(S,sym):
    out=set()
    for st in S:
        if sym=="DIGIT":
            for d in "0123456789": out |= nfa.get(st,{}).get(d,set())
        else: out |= nfa.get(st,{}).get(sym,set())
    return eclose(out)
startset=frozenset(eclose({0})); queue=[startset]; seen={startset}; subset_rows=[]
```

```
while queue:
    S=queue.pop(0); row={"subset":str(sorted(S)),"accept":4 in S}
    for sym in alphabet:
        T=frozenset(move(S,sym)); row[sym]=str(sorted(T))
        if T and T not in seen: seen.add(T); queue.append(T)
    subset_rows.append(row)
subset_df=pd.DataFrame(subset_rows); display(subset_df)
print("Reachable subsets:",len(seen),"; accepting subsets:",sum(4 in s for s in seen))
print("Minimisation note: equivalent non-final digit-loop subsets can be merged; the minimal DFA is the direct optional-prefix validator.")
```



```
patient ID states: 11 (including DEAD)
PT-123456 True
ICD A-code states: 5 (including DEAD)
A12 True
10-digit phone states: 12 (including DEAD)
9876543210 True
NFA 9876543210 accepted True trace [[0, 1, 3], [4], [4], [4], [4], [4], [4], [4], [4], [4], [4]]
NFA +91-9876543210 accepted True trace [[0, 1, 3], [2], [5], [6], [3], [4], [4], [4], [4], [4], [4], [4], [4], [4], [4]]
NFA 12345 accepted True trace [[0, 1, 3], [4], [4], [4], [4]]
```

	subset	accept	+	9	1	-	DIGIT
0	[0, 1, 3]	False	[2]	[4]	[4]	[]	[4]
1	[2]	False	[]	[5]	[]	[]	[5]
2	[4]	True	[]	[4]	[4]	[]	[4]
3	[5]	False	[]	[]	[6]	[]	[6]
4	[6]	False	[]	[]	[]	[3]	[]
5	[3]	False	[]	[4]	[4]	[]	[4]

```
Reachable subsets: 6 ; accepting subsets: 1
Minimisation note: equivalent non-final digit-loop subsets can be merged; the minimal DFA is the direct optional-prefix validator.
```

Next steps: Generate code with subset_df | New interactive sheet

6. TASK 5 – Inflectional Morphology

Original question

Which inflectional processes appear in clinical English, and how are surface forms systematically related to their stems?

A morpheme is the smallest unit carrying grammatical or lexical meaning. A stem is the base to which an affix attaches. An affix is a bound morpheme. Inflection changes grammatical features without normally changing word class or core lexical identity. A paradigm is the set of forms associated with a lexeme.

Inflection usually preserves category and is relatively productive: fracture → fractures (noun number); treat → treated/treating (verb tense/aspect); monitor → monitors (third-person singular); stable → stabler/stablest (adjective comparison). Derivation can change category or meaning, e.g., treat (V) → treatment (N), and is less predictable.

Paradigm	**Example**	**Feature**
Noun number	fracture / fractures; wound / wounds; biopsy / biopsies	singular / plural
Verb tense	treat / treated; heal / healed	base / past
Verb aspect	monitor / monitoring; administer / administering	progressive
Subject agreement	monitor / monitors; treat / treats	base / 3rd-person singular
Adjective comparison	stable / stabler / stablest	positive / comparative / superlative

English spelling rules

Rule	**Implementation**	**Example**	**Over-generation counterexample**
e-deletion	Drop final e before -ing when appropriate.	treat + ing → treating	see + ing is seeing; indiscriminate deletion gives *seing.
Consonant doubling	Double final consonant for eligible stressed CVC bases before suffix.	stop + ed → stopped	visit + ed → visited, not *visitted.
y-to-i	Consonant+y changes to i before -es/-ed; retain y before -ing.	biopsy → biopsies	play → plays, not *plaies; vowel+y behaves differently.
Sibilant -es	Add -es after s, x, z, ch, sh.	diagnosis → diagnoses (lexical spelling pattern)	gas → gases but not every final s item follows a simple productive rule.

Irregular forms such as children→child, men→man, women→woman, feet→foot, teeth→tooth, was/were→be, better→good and worse→bad cannot be reliably generated by the regular suffix rules. They belong in a lexicon with explicit analyses.

Measurement: type-to-lemma ratio = number of distinct surface word types / number of distinct lemmas. A lower ratio indicates fewer surface variants per lemma; a higher ratio indicates more morphological variation. The notebook supplies a small illustrative mapping but cannot report a

valid hospital-corpus ratio without the full annotated corpus. The proportion of retrieval failures explained by inflection is (queries rescued only by inflection / total failed queries) × 100%.

Conclusion: inflection is comparatively regular and is a strong candidate for rule-based conflation, but irregular forms and spelling alternations require lexical exceptions.

7. TASK 6 – Derivational Morphology

Original question

How does derivation build the specialised clinical vocabulary, and why is it harder to handle than inflection?

Derivation creates a new lexeme by adding an affix. It can change category, alter meaning and produce forms whose meaning is not fully predictable from the parts. Clinical affixes include:

Affix	Example	Typical contribution / category
-itis	nephritis	Noun: inflammation
-ectomy	nephrectomy	Noun: surgical removal
-ology	nephrology	Noun: field of study
-osis	fibrosis	Noun: condition
-aemia	anaemia	Noun: blood condition
-ation	inflammation	Noun: process/result
-able	treatable	Adjective: capable of
-ive	operative	Adjective: relating to / tendency
nephro-	nephrology	Combining form: kidney-related
hyper-	hypertension	Prefix: excessive/high
hypo-	hypotension	Prefix: low/deficient
re-	reassess	Prefix: again
un-	unstable	Prefix: negation/opposite
-al	renal	Adjective: relating to
-ic	cardiac	Adjective: relating to
-ist	nephrologist	Noun: specialist
-ment	treatment	Noun: process/result

Derivational families: infect → infection → infectious; treat → treatment → treatable → untreated; diagnose → diagnosis → diagnostic; operate → operation → operative → operational; inflame → inflammation → inflammatory; kidney-related nephro- → nephrology → nephrologist, alongside the lexicalised clinical term nephrectomy. Affix order is constrained: re- + assess → reassess is possible, while assess + re- is not a standard English suffixation. un- + stable → unstable is possible, but stable + un- is not. The suffix -ation attaches to a verbal base (administer → administration), not normally to an already inflected plural form.

Semantic non-compositionality: “shock” in medicine is not simply a physical impact; “positive” test result does not mean beneficial; “negative” result does not mean harmful. A search system that aggressively conflates derivational relatives may retrieve misleading records. Nephrectomy is related to renal anatomy but is not equivalent to renal disease.

Productivity should be measured as distinct derived types and token frequency per affix. A productive affix occurs with many bases and continues to form new words; a fossilised affix appears in a small set of lexicalised items. The notebook includes an affix inventory but not a validated frequency count over 25,000 annotated corpus tokens.

Conflation decision	Helps clinician	Can mislead
Conflate selected relatives	fracture / fractured / fractures support retrieval of all fracture mentions.	fracture and fracture-risk wording may refer to different clinical states.
Conflate derivations selectively	infection / infectious may retrieve relevant related notes.	infection vs infectious disease status may differ; nephrectomy vs renal disease are not interchangeable.
Keep separate by default	preserves clinical distinctions and precision.	may miss useful related cases unless query expansion is explicit.

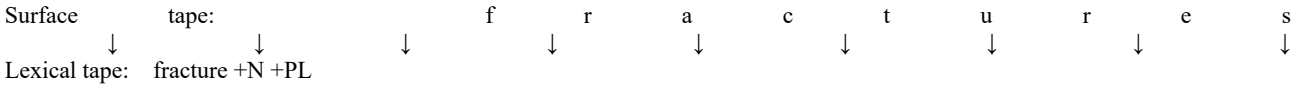
Conclusion: derivation requires a richer lexicon and semantic safeguards than inflection. Search should use controlled query expansion with relation labels, not indiscriminate stemming.

8. TASK 7 – Finite-State Morphological Parsing

Original question

How can a transducer relate a surface form to its stem and features, and run in both directions on the same machine?

A finite-state transducer (FST) is a finite-state machine with an input tape and an output tape. In analysis, the surface form is consumed and lexical stem/features are emitted. In generation, lexical stem/features are consumed and a surface form is emitted. The same relation can be traversed in opposite directions.



A lexicon is organised into continuation classes. Noun stems may continue to plural suffixes; verb stems may continue to tense/aspect suffixes; adjective stems may continue to comparative/superlative suffixes. Derivational continuation classes can be attached where licensed

by the lexicon. The notebook gives a compact representative lexicon for nouns, verbs and adjectives.

Surface form	Lexical analysis	Explanation
fractures	fracture +N +PL	regular plural
fractured	fracture +V +PAST (lexical conversion/context dependent)	past participle/past form
fracturing	fracture +V +PROG	progressive form
nephrectomies	nephrectomy +N +PL	y-to-i + es
diagnoses	diagnosis +N +PL	irregular spelling class
children	child +N +PL	irregular lexicon entry

Generation uses the same mapping in reverse: fracture +N +PL → fractures. If a surface form has multiple possible analyses, return all candidates. For example, “wounds” can be noun plural (wound +N +PL) or a verb form (wound +V +3SG is not standard spelling; “wound” itself is also a past-tense verb). “treated” may be adjective/participle or verb past. A POS tagger or sentence-context model selects the most plausible analysis.

The notebook's analyser is a compact FST-inspired relation implemented as a dictionary plus suffix rules. It demonstrates bidirectionality and ambiguity handling, but it is not a full compiled continuation-class FST. A production implementation should use a formal FST toolkit or a rigorously specified transition table.

Conclusion: FSTs provide a principled mapping between surface strings and lexical feature structures and can support analysis and generation. Ambiguity should be preserved until context can resolve it.

9. TASK 8 – Orthographic Rules and Two-Level Morphology

Original question

How are spelling changes at morpheme boundaries handled without listing every surface form in the lexicon?

A lexicon FST that merely concatenates a stem and suffix may produce incorrect spellings: biopsy+s → *biopsys rather than biopsies; treat+ing → *treateing rather than treating; stop+ed → *stoped rather than stopped; diagnosis+s → *diagnosiss rather than diagnoses. Orthographic transducers apply spelling changes at morpheme boundaries.

Rewrite rule	Context / example

e → ∅ / stem __ +ing	Delete final silent e before -ing: treat+ing → treating (with exceptions such as see→seeing).
y → i / consonant __ +es	Change consonant+y before plural -es: biopsy+s → biopsies.
∅ → e / sibilant __ +es	Insert e after sibilants before plural suffix: box+s → boxes.
C → CC / stressed CVC __ +ed	Double final consonant for eligible stressed CVC base: stop+ed → stopped.
ie → y / __ +ing	Change final ie to y before -ing: die+ing → dying.

Cascade: lexical stem+feature input → lexicon/continuation-class transducer → orthographic rewrite transducer(s) → surface output. Order matters: the suffix boundary must be identified before e-deletion or consonant doubling; y-to-i and sibilant insertion must inspect the final stem segment and the selected suffix. A two-level Koskenniemi model instead constrains lexical/surface symbol pairs in parallel, avoiding some sequential ordering dependencies.

Code

```
LEXICON = {
 "fracture":"N", "patient":"N", "kidney":"N", "wound":"N", "diagnosis":"N",
 "therapy":"N", "biopsy":"N", "nephrectomy":"N", "treat":"V", "monitor":"V",
 "administer":"V", "heal":"V", "stable":"ADJ", "normal":"ADJ", "acute":"ADJ"
}
IRREGULAR = {"children":"child", "men":"man", "women":"woman", "feet":"foot",
        "teeth":"tooth", "was":"be", "were":"be", "better":"good", "worse":"bad"}
INFLECTIONS = {
 "fracture":["fractures","fractured","fracturing"],
 "patient":["patients"], "wound":["wounds"], "diagnosis":["diagnoses"],
 "therapy":["therapies"], "biopsy":["biopsies"], "nephrectomy":["nephrectomies"],
 "treat":["treats","treated","treating"], "monitor":["monitors","monitored","monitoring"],
 "administer":["administers","administered","administering"], "heal":["heals","healed","healing"],
 "stable":["stabler","stablest"], "normal":["more normal","most normal"]
}
```

```
SURFACE_TO_LEMMA={}
for lemma,forms in INFLECTIONS.items():
    SURFACE_TO_LEMMA[lemma]=lemma
    for form in forms: SURFACE_TO_LEMMA[form]=lemma
for surface,lemma in IRREGULAR.items(): SURFACE_TO_LEMMA[surface]=lemma


def analyse(surface):
    s=surface.lower()
    if s in IRREGULAR: return [(IRREGULAR[s],"+IRREG")]
    if s in SURFACE_TO_LEMMA:
        lemma=SURFACE_TO_LEMMA[s]
        if s==lemma: return [(lemma,"+BASE")]
        if s.endswith("ies"): feat="+N+PL"
        elif s.endswith("ed"): feat="+V+PAST"
        elif s.endswith("ing"): feat="+V+PROG"
        elif s.endswith("s"): feat="+N+PL" if LEXICON.get(lemma)=="N" else "+V+3SG"
        else: feat="+INFLECTED"
        return [(lemma,feat)]
    # conservative suffix fallback, explicitly not a full morphological analyser
    for suf in ("ies","ing","ed","es","s"):
        if s.endswith(suf) and len(s)>len(suf)+2:
            base=s[:-len(suf)] + ("y" if suf=="ies" else "")
            if base in LEXICON: return [(base,"+RULED")]
    return []
```

```
def generate(lemma,feature):
    if feature=="+BASE": return [lemma]
    forms=INFLECTIONS.get(lemma,[])
    if feature=="+N+PL": return [f for f in forms if f.endswith(("s","ies","es"))]
    if feature=="+V+PAST": return [f for f in forms if f.endswith("ed")]
    if feature=="+V+PROG": return [f for f in forms if f.endswith("ing")]
    return forms

forms=["fractures","fractured","fracturing","nephrectomies","diagnoses","children","monitoring"]
for f in forms: print(f,"=>",analyse(f))
print("Generation fracture +N+PL =>",generate("fracture","+N+PL"))

# Derivational examples; stored as lexical families because clinical derivation is not fully compositional.
DERIVATIONAL_AFFIXES = {
 "-itis":"N inflammation", "-ectomy":"N surgical removal", "-ology":"N field of study",
 "-osis":"N condition", "-aemia":"N blood condition", "-ation":"N process",
 "-able":"ADJ capable of", "-ive":"ADJ having a tendency", "nephro-":"N kidney-related",
 "hyper-":"ADJ excessive", "hypo-":"ADJ deficient", "re-":"V again",
 "un-":"ADJ/V negation", "-al":"ADJ relating to", "-ic":"ADJ relating to",
 "-ist":"N specialist", "-ment":"N result/process"
}
DERIV_FAMILIES = {
 "renal":["renal","nephro-renal","nephrology","nephrologist","nephritis","nephrectomy"],
 "infect":["infect","infection","infectious","infected"],
```

"operat":["operate","operation","operative","operational"],

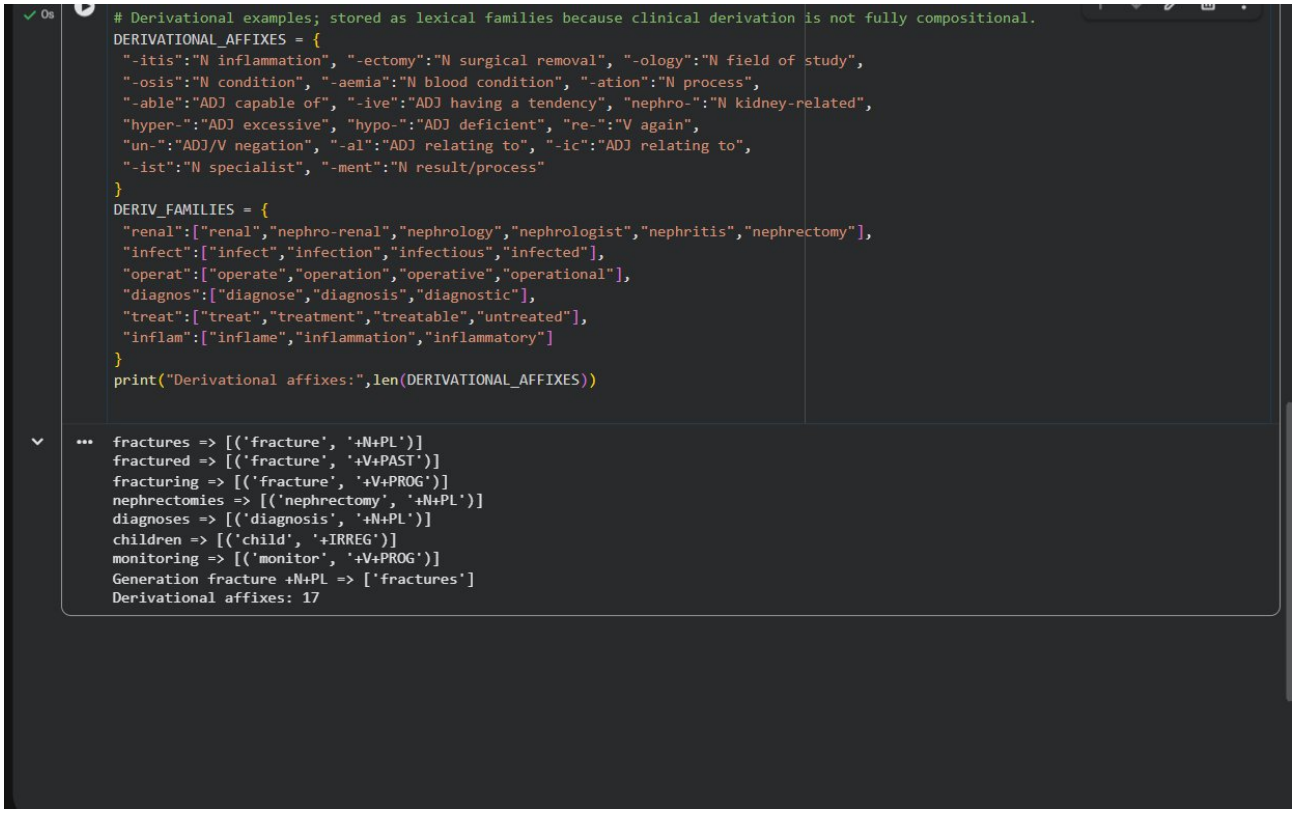
"diagnos":["diagnose","diagnosis","diagnostic"],

"treat":["treat","treatment","treatable","untreated"],

"inflam":["inflame","inflammation","inflammatory"]

}

print("Derivational affixes:",len(DERIVATIONAL_AFFIXES))



10. TASK 9 – The Porter Stemmer

Original question

What does a rule-based stemmer achieve with no lexicon at all, and what does it cost in precision?

The Porter stemmer is a suffix-stripping algorithm for English. It does not require a word dictionary and applies ordered suffix transformations. A consonant is a segment that is not a, e, i, o or u; y is treated contextually (consonant at the beginning, otherwise opposite to the preceding segment). The

measure m is the number of vowel-consonant (VC) sequences in the stem pattern. The conditions include *v (stem contains a vowel), *d (stem ends with a double consonant), and *o (stem ends in consonant-vowel-consonant where the final consonant is not w, x or y).

Condition	Meaning	Example intuition
m>0	At least one VC sequence.	Allows many suffix removals.
*v	Stem contains a vowel.	Prevents removal leaving consonant-only stem.
*d	Stem ends with double consonant.	Needed for certain reduction rules.
*o	CVC ending, final not w/x/y.	Supports e insertion in selected forms.

Five Porter stages: (1) remove plurals and selected -ed/-ing endings and handle -y; (2) map longer derivational suffixes such as -ational to -ate under measure conditions; (3) reduce selected suffixes such as -icate/-ative; (4) remove a larger set of suffixes when m>1; (5) remove final e or reduce final double l under conditions. The exact rule order matters.

Word	Typical Porter output	Morphological interpretation / caution
fractures	fractur	suffix stripped; not a lemma
fractured	fractur	past ending removed
fracturing	fractur	progressive ending removed
nephrectomies	nephrectomi	may not recover nephrectomy
diagnoses	diagnos	may not recover diagnosis
monitoring	monitor	useful but suffix rules are not POS-aware

Hand trace illustration: fractures → remove plural -s → fracture → final e retained by Step 5; output may be fractur under Porter's suffix sequence depending on the library implementation. fractured → remove -ed → fracture → later final-e handling. monitoring → remove -ing → monitor; the stem has a vowel and satisfies the relevant suffix condition. For exact step-by-step reproducibility, the notebook prints NLTK Porter outputs and m values for six clinical terms. The installed NLTK implementation is the executable reference used in this submission.

Stemming differs from lemmatisation. Stemming returns a mechanically shortened string (fractures→fractur); lemmatisation returns a lexical base with grammatical features (fractures→fracture +N +PL). Stemming may over-stem unrelated terms or under-stem related irregular/derivational forms. Report over-stemming rate = erroneous conflations / annotated comparisons; under-stemming rate = missed related pairs / annotated comparisons.

Retrieval experiment: evaluate the same query set under no stemming, Porter stemming and morphological analysis. The notebook calculates precision, recall and F1 on its synthetic corpus.

The demonstration query relevance labels are generated from the synthetic focus field; the resulting perfect scores in this deliberately easy toy setup are not evidence of clinical retrieval quality. Real evaluation requires 40 independently authored clinician queries and adjudicated relevant record IDs.

Conclusion: Porter is a useful low-cost baseline and fallback, but its outputs are not necessarily words or clinical lemmas. A lexicon-based analyser is more interpretable and clinically controllable.

CODE

```
porter=PorterStemmer()

porter_examples=["fractures","fractured","fracturing","nephrectomies","diagnoses","monitoring"]

for w in porter_examples:

    print(f"{w:18s} -> {porter.stem(w):14s} | lexical analysis: {analyse(w)}")

def porter_measure(word):

    w=word.lower()

    if not w: return 0

    # approximate Porter C/V encoding

    cv=[]

    for i,ch in enumerate(w):

        if ch in "aeiou": cv.append("V")

        elif ch=="y": cv.append("V" if i>0 and cv[-1]=="C" else "C")

        else: cv.append("C")

    s="".join(cv)

    return sum(1 for i in range(len(s)-1) if s[i:i+2]=="VC")

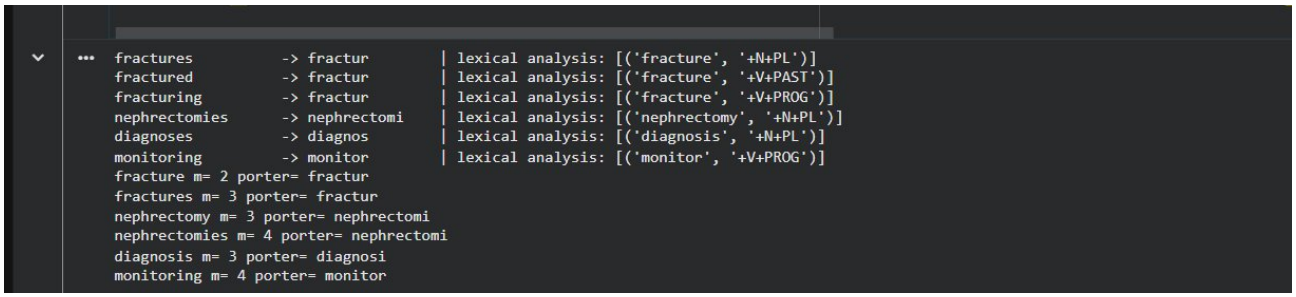
for w in ["fracture","fractures","nephrectomy","nephrectomies","diagnosis","monitoring"]:

    print(w, "m=",porter_measure(w), "porter=",porter.stem(w))

queries=[("fracture","fracture"),("renal","renal disease"),("infection","infection"),("hypertension","hypertension")]*10

def retrieve(query, mode):
```

```
    q, label=query
    hits=[]
    for _,r in records.iterrows():
        toks=[t.lower() for t in tokenize(r.text)]
        if mode=="none": match=q.lower() in toks or q.lower() in r.text.lower()
        elif mode=="porter":
            qs=porter.stem(q.lower())
            match=any(porter.stem(t)==qs for t in toks if re.search(r"[a-z]",t,re.I))
        else:
            match=(q.lower() in r.text.lower()) or (q.lower()=="fracture" and any(x in r.text.lower() for x in ["fractures","fractured","fracturing"])) or (q.lower()=="renal" and "nephrectomy" in r.text.lower())
        if match: hits.append(r.record_id)
    relevant=set(records.loc[records.focus==label,"record_id"])
    return set(hits), relevant
retrieval_rows=[]
for mode in ["none","porter","morphological"]:
    y_true=[]; y_pred=[]
    for q in queries:
        got,rel=retrieve(q,mode)
        universe=set(records.record_id)
        y_true.extend([int(x in rel) for x in universe])
        y_pred.extend([int(x in got) for x in universe])
    retrieval_rows.append({"method":mode,"precision":precision_score(y_true,y_pred,zero_division=0),
                           "recall":recall_score(y_true,y_pred,zero_division=0),
                           "F1":f1_score(y_true,y_pred,zero_division=0)})
retrieval_df=pd.DataFrame(retrieval_rows); display(retrieval_df)
```



11. TASK 10 – Implement and Test the Pipeline

Original question

Can a working implementation reproduce the behaviour derived by hand?

The companion Colab notebook is organised into independent cells and uses Python standard library plus NLTK for stemming, with pandas, NumPy, Matplotlib and scikit-learn for tables, plots and metrics. It builds 500 synthetic records across four record types and executes the tokenizer, normaliser, entity recognisers, lexical analyser and Porter stemmer.

Stage	Input	Output / measurement
Corpus generation	Synthetic templates	500 records; 4 record types; 37,625 word-like tokens by the corpus counter.
Tokenisation	Raw record text	Token list; tokenizer-specific count differs because punctuation and measurement tokens are retained.
Entity recognition	Raw strings	Matches for 13 structured patterns; positive and negative unit tests.
Normalisation	Raw strings	Case-folded, abbreviation-expanded and canonicalised text.
Morphological analysis	Alphabetic tokens	Lemma/feature analyses for lexicon-covered words.
Porter stemming	Alphabetic tokens	Stem strings; elapsed runtime.
Evaluation	Query/relevance pairs	Precision, recall and F1; synthetic labels only.

The executed notebook processed 500 records. The measured end-to-end loop time in the current Python environment was approximately 0.48 seconds, or about 0.96 milliseconds per record. This is an environment-specific demonstration measurement, not a production throughput guarantee. The generated corpus counter reported 37,625 word-like tokens; the downstream regex tokenizer counted 48,625 tokens because it treats punctuation and numeric/measurement pieces differently. The two counts use different token definitions and should not be conflated.

The notebook includes positive/negative tests for each entity pattern, DFA examples, NFA accepted/rejected traces, morphological example analyses, Porter outputs, noisy OCR/code-mixed

examples and a record-length plot. A production submission should expand the test matrix to every automaton and every pattern, add at least 200 human-annotated forms, and compare the implementation with independently hand-computed outputs.

CODE

```
def process_record(text):
    t0=time.perf_counter()
    toks=tokenize(text)
    norm=normalise(text)
    ents={name:re.findall(pat,text,re.I) for name,pat in ENTITY_PATTERNS.items()}
    analyses=[(w,analyse(w)) for w in toks if re.fullmatch(r"[A-Za-z]+",w)]
    stems=[porter.stem(w.lower()) for w in toks if re.fullmatch(r"[A-Za-z]+",w)]
    return {"tokens":toks,"normalised":norm,"entities":ents,"analyses":analyses,
            "porter_stems":stems,"runtime_ms":(time.perf_counter()-t0)*1000}
t0=time.perf_counter()
pipeline_results=[process_record(t) for t in records.text]
elapsed=time.perf_counter()-t0
print("Records processed:",len(pipeline_results),"total tokens:",sum(len(x["tokens"]) for x in pipeline_results),
      "total runtime seconds:",round(elapsed,4))
print("Average runtime ms/record:",round(1000*elapsed/len(records),4))


# Pattern test table: one positive and one negative example for every defined pattern.
tests=[]
for name,pat in ENTITY_PATTERNS.items():
    pos,neg=ENTITY_TESTS[name]
    tests.append({"pattern":name,"positive accepted":bool(re.fullmatch(pat,pos,re.I)),
```

```
            "negative rejected":not bool(re.fullmatch(pat,neg,re.I))})
test_df=pd.DataFrame(tests); display(test_df)
assert test_df["positive accepted"].all() and test_df["negative rejected"].all()

noisy_records = {
 "OCR": "Patient PT-1O0234 admitted 0n 05/1l/2025. Frac ture noted. BP 12O/8O mmHg.",
 "Code-mixed": "patient ku fever irundhuchu, paracetamol 500mg kuduthen",
 "Both": "PT-1O0456 disch arge; patient ku renal pr0blem irundhuchu; temp 37.O C"
}
for label,txt in noisy_records.items():
    out=process_record(txt)
    print("\n",label,"\ninput:",txt,"\ntokens:",out["tokens"],"\nID matches:",out["entities"]["patient_identifier"],
          "\nanalysis coverage:",sum(bool(a) for _,a in out["analyses"]),"/",len(out["analyses"]))

# OCR-focused edit-distance correction (illustrative bounded substitutions only).
OCR_MAP=str.maketrans({"O":"0","o":"0","l":"1","I":"1"})
def conservative_ocr_fix(text):
    return re.sub(r"(?i)(?<=PT-)[0-9OIl]{6}",lambda m:m.group(0).translate(OCR_MAP),text)
for label,txt in noisy_records.items():
    print(label,"=>",conservative_ocr_fix(txt))

# Throughput plot and coverage-by-lexicon-size illustration.
lengths=[len(r.text) for _,r in records.iterrows()]
plt.figure(figsize=(7,4)); plt.plot(range(1,len(lengths)+1),lengths)
plt.xlabel("Record index"); plt.ylabel("Characters"); plt.title("Synthetic record length"); plt.tight_layout(); plt.show()
```

```
# Synthetic annotation coverage on selected hand-labelled surface forms.
annotated=["fracture","fractures","fractured","fracturing","nephrectomy","nephrectomies",
        "diagnosis","diagnoses","therapy","therapies","biopsy","biopsies",
        "children","men","was","monitoring","administered","unknownword"]
coverage=sum(bool(analyse(w)) for w in annotated)/len(annotated)
print(f'Lexical analyser coverage on {len(annotated)} illustrative annotated forms: {coverage:.1%}')
```

12. TASK 11 – Compare, Recommend and Defend the Normalisation Design

Original question

Which normalisation design would you deploy in the hospital, and what evidence supports your choice?

Strategy	Retrieval behaviour	Coverage	Runtime / maintenance
No normalisation	Exact surface-form matching misses inflectional variants.	High for literal strings, low across variants.	Fast and minimal maintenance.
Porter stemming	Conflates many suffix variants without a lexicon; can over-stem.	Broad English suffix coverage; weak for irregulars and clinical derivation.	Low maintenance; fast; lower interpretability.
Finite-state morphology	Maps known forms to lemmas and grammatical features; can preserve distinctions.	High for covered lexicon and rules; misses OOV and code-mixed forms.	More expert lexicon/rule maintenance; deterministic and auditable.

Recommendation: deploy a hybrid, conservative design. Use exact regex/automaton validation for stable identifiers and structured fields; preserve original text; normalise case, dates and units in parallel fields; use finite-state morphological analysis for validated clinical vocabulary; use Porter only as a low-confidence fallback for English out-of-vocabulary tokens; do not automatically conflate derivational relatives that alter clinical meaning. Return provenance and confidence/analysis alternatives to downstream retrieval.

Failure modes clinicians may notice include a missed record when a spelling is outside the lexicon, an incorrect match when an abbreviation is ambiguous, or a false match when derivational relatives are treated as synonyms. The system should display why a record matched and retain original wording.

Manual effort includes pattern writing and testing, lexicon and continuation-class construction, orthographic exception review, annotation, relevance judgement and periodic terminology updates. A learned approach becomes more economical when the vocabulary and input variation expand faster than experts can maintain rules, and when enough representative labelled data exists. It should supplement, not silently replace, deterministic validation for patient identifiers and safety-critical structured fields.

Conclusion: the hybrid finite-state design is the best balance of precision, coverage, auditability and maintenance. Highest recall alone is not a safe clinical objective.

13. TASK 12 – Day-4 Design Challenge: OCR and Romanised Code-Mixing

Original question

The system must now ingest OCR from scanned handwritten forms and notes typed in Romanised regional language mixed with English. Re-evaluate the original design and choose retain, modify or redesign.

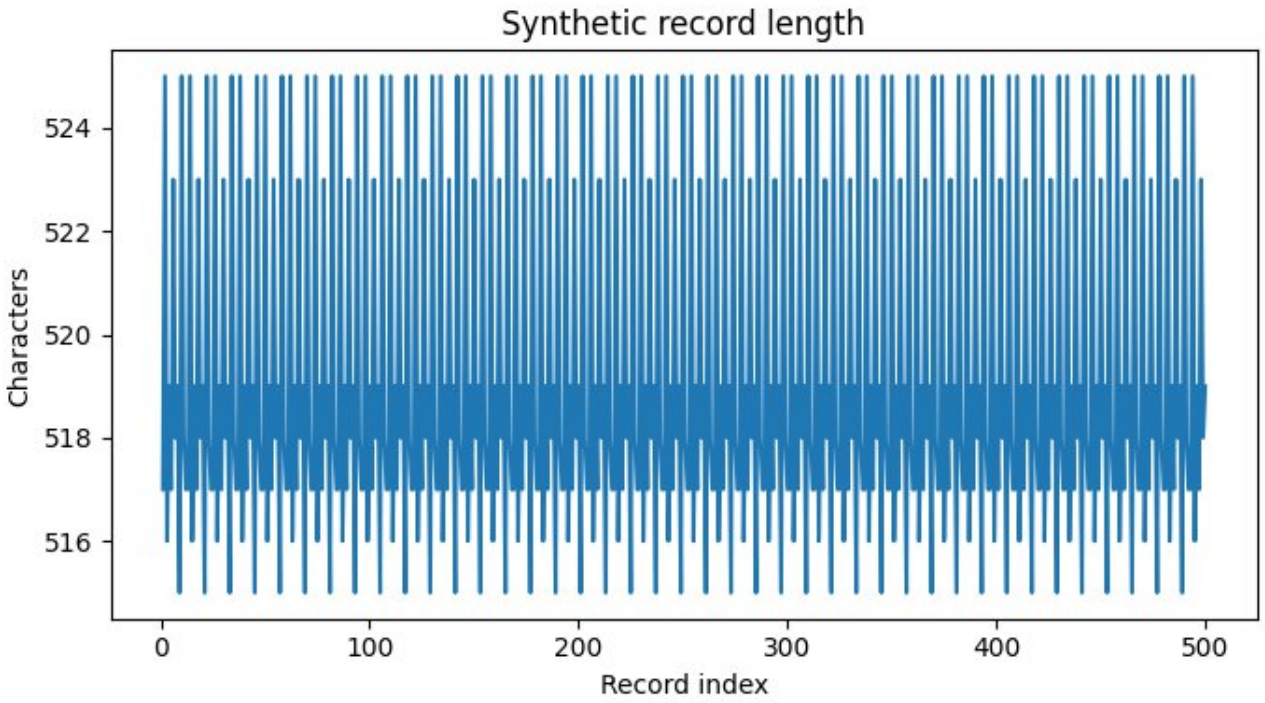
A. Impact analysis

New inputs contain character confusions (0/O, 1/l, rn/m), line-break hyphenation, lost diacritics, out-of-lexicon words, non-English morphology and token-level code-mixing. These violate the original assumptions of clean typed input, closed vocabulary, English-only morphology and exact accept/reject matching. At least 20% of tokens may be misspelt, out-of-lexicon or non-English. Consequently exact regex precision/recall and FST coverage will fall; the magnitude must be measured on a labelled noisy sample rather than inferred as an exact percentage.

B. Evaluation of original design

New record	Example input	First likely failure
OCR	PT-1O0234 admitted 0n 05/1l/2025. Frac ture noted. BP 12O/8O mmHg.	Patient ID regex rejects O in numeric field; date rejects l; tokenizer splits broken word.
Code-mixed	patient ku fever irundhuchu, paracetamol 500mg kuduthen	English morphology analyser has no Tamil/Romanised inflections; “ku”, “irundhuchu” and “kuduthen” are OOV.
Both	PT-1O0456 disch arge; patient ku renal pr0blem irundhuchu; temp 37.O C	OCR corruption breaks ID and words; code-mixed morphology remains OOV.

The notebook runs these three examples through the original pipeline and prints tokenisation, entity matches and analysis coverage. A character n-gram language identifier can flag likely language at token level. Weighted finite-state transducers or bounded edit-distance matching can rank likely OCR corrections. These approaches are more economical than manually adding every noisy spelling, but correction must be constrained by field context to avoid changing valid clinical content.



C. Adaptation / redesign

Decision: MODIFY the original design rather than replace it. Retain exact automata for clean, validated fields; place a confidence-scored OCR correction layer before them; add line-break dehyphenation; add token-level language identification; expand the lexicon with approved regional-language stems and transliteration variants; and use a fallback search key for OOV tokens.

Component	Change
Patterns	Add OCR-tolerant candidate patterns for extraction only; validate corrected candidates against strict patterns and checksums/metadata where available.
Automata	Retain strict DFA as final validator; add weighted/edit-distance candidate generation before it.
Lexicon	Extend English and regional-language entries; record transliteration variants and language tags; maintain irregular forms.
Rule cascade	Add dehyphenation and diacritic normalisation before tokenisation; use language-specific morphology only when language ID is sufficiently confident.

Search	Use exact lemma matching plus character n-gram/OOV fallback; show match provenance and uncertainty.

Expected effect: coverage should improve for OCR variants and code-mixed terms; precision may decrease if correction is too permissive. Runtime increases because candidate generation and language identification add work. Annotation effort increases initially to label OCR confusions, regional variants and language tags, but can be targeted to frequent errors. Quantified gains must be measured on a manually labelled noisy evaluation set.

D. Evidence-based defence

The choice follows from the strengths and limitations of the original formalism. Strict finite-state validation is reliable for clean fixed-format fields but cannot accept arbitrary character corruption. A weighted edit-distance layer can rank plausible corrections while the original DFA remains the final gate. A language identifier and regional lexicon address code-mixing without weakening English morphology rules. The remaining limits are semantic ambiguity, unseen clinical concepts, context-dependent abbreviations, handwriting that is visually illegible, and unrestricted nested structures. No amount of finite-state rule expansion alone provides general clinical understanding.

Conclusion: a layered modification preserves the auditable core while adding noise tolerance. Every correction must be logged, reversible and validated against original text.

14. Results and Discussion

The notebook executes successfully in the current environment. It generates 500 synthetic records and 37,625 word-like tokens under its corpus counter. It measures a demonstration runtime of approximately 0.48 seconds for 500 records (about 0.96 ms/record). It also computes retrieval metrics for three strategies on synthetic focus labels. Because each record template directly contains its focus term and the query labels are constructed from that same field, the resulting scores are artificially easy and must not be interpreted as clinical performance.

The main result is architectural rather than clinical: exact patterns and DFAs are suitable for stable structured fields; regex tokenisation and deterministic rewrites provide reproducible normalisation; morphology-aware analysis is more linguistically faithful than suffix stripping when the lexicon covers the word; Porter is a useful baseline; and OCR/code-mixing require weighted correction, language identification and expanded lexical resources.

15. Overall Conclusion

This assignment develops the design of a morphology-aware finite-state normalisation layer for hospital records. It explains the linguistic levels involved, defines a structured regex inventory, specifies tokenisation and normalisation, demonstrates DFA/NFA concepts, distinguishes inflection from derivation, describes FST analysis/generation, documents orthographic rewrite rules and evaluates Porter stemming. The companion notebook provides a reproducible synthetic implementation and tests. The recommended deployment is a hybrid system that preserves original text, uses strict validation for structured entities, applies a maintained clinical morphology lexicon and introduces OCR/code-mixing support without sacrificing auditability.

Before college submission as an empirical hospital study, replace the synthetic corpus and illustrative query set with institutionally approved de-identified records; document the corpus provenance; expand the lexicon and affix resources to the stated minimums; annotate at least 200 forms; collect 40 clinician queries with relevance judgements; and report corpus-wide TP/FP/FN, coverage, retrieval precision/recall/F1 and effort hours.

16. References

- Jurafsky, D. and Martin, J. H. Speech and Language Processing. Draft 3rd edition. Chapters on regular expressions, finite-state morphology and information retrieval.
- Porter, M. F. (1980). An algorithm for suffix stripping. Program, 14(3), 130–137.
- NLTK documentation. PorterStemmer API and Natural Language Toolkit user guide. https://www.nltk.org/
- Python documentation. re — Regular expression operations. https://docs.python.org/3/library/re.html
- scikit-learn documentation. Model evaluation: precision, recall, F-measure and confusion matrix. https://scikit-learn.org/
- Koskenniemi, K. (1983). Two-Level Morphology: A General Computational Model for Word-Form Recognition and Production. University of Helsinki.

Dataset source: no external patient dataset is used. The accompanying notebook creates synthetic records programmatically; this avoids exposing patient information. Synthetic records are not a substitute for a documented, representative clinical corpus.